

数据库 SQL

今年预测题（5道）· 含作答区域 · 附详细答案

预测依据：三年历年真题考点统计 + 题型轮换规律分析

题号	数据库场景	核心考点	分值	概率
第1题	教学数据库	多表JOIN + 子查询 + GROUP BY	25分	★★★★ 极高
第2题	医院数据库	聚合+HAVING + 日期函数 + 子查询	25分	★★★★ 极高
第3题	商店数据库	UPDATE + DELETE + 条件子查询	25分	★★★☆☆ 高
第4题	教学数据库	排名函数 DENSE_RANK + 多重聚合	25分	★★★☆☆ 高
第5题	自选/综合	INTERSECT / EXISTS + 复杂子查询	25分	★★★☆☆ 中高

使用说明：试题页含完整题目和作答空白框，请先独立完成。答案页从第二部分开始，每题附完整 SQL 代码和得分要点解析。

第一部分 预测题目（请先独立完成，再翻看答案）

数据库模式速查（答题前请先熟悉表结构）

教学数据库

Student(SId, Sname, Sage, Ssex)

Course(CId, Cname, TId)

Teacher(TId, Tname)

SC(SId, CId, score)

医院数据库

Doctor(Cno, Dno, Cname)

Department(Dno, Dname)

Patient(Pno, Pname, Age)

Treatment(Cno, Pno, Tdate)

商店数据库

Employee(EmployeeID, Name, Gender, Salary, StoreID)

Member(MemberID, Name)

Store(StoreID, StoreName, EmployeeCount)

Membership(MemberID, StoreID)

第一题 教学数据库·多表查询

(25分)

对于教学数据库的四个基本表，试写出下列操作的 SQL 语句（每小题 5 分）：

1. 查询在 SC 表中有成绩记录的学生信息（学号、姓名、性别）。

第1小题作答区域

2. 查询平均成绩大于等于 60 分的同学的学号、姓名和平均成绩，按平均成绩降序排列。

第2小题作答区域

3. 查询学过"张三"老师所教所有课程的学生姓名，去除重复。

第3小题作答区域

4. 查询有两门及以上课程不及格（score < 60）的学生学号、姓名及其所有课程的平均成绩。

第4小题作答区域

5. 查询学生的总成绩并进行排名，总分重复时不保留名次空缺（即并列后继续顺序编号）。

第5小题作答区域

第二题 医院数据库 · 聚合与日期

(25分)

对于医院数据库的四个基本表，试写出下列操作的 SQL 语句（每小题 5 分）：

1. 查询编号为 c3 的医生所治疗的所有患者姓名（用 JOIN 或子查询均可）。

第1小题作答区域

2. 将医生编号为 c4 的医生调换到科室编号为 d5 的科室。

第2小题作答区域

3. 删除患者姓名为"张三"的患者的所有诊疗记录。

第3小题作答区域

4. 查询每个科室的患者平均年龄，只输出平均年龄大于 20 岁的科室编号、科室名称和平均年龄，按平均年龄升序排列。

第4小题作答区域

5. 查询诊疗表中就诊日期与 2025 年相差年份最大的记录，显示医生编号、患者编号、就诊日期和相差年份。

第5小题作答区域

第三题 商店数据库·更新与删除

(25分)

对于商店数据库的四个基本表，试写出下列操作的 SQL 语句（每小题 5 分）：

1. 查询同时属于店铺 s1 和店铺 s2 的会员编号。

第1小题作答区域

2. 把所有女性员工的工资增加 10%。

第2小题作答区域

3. 查询工资高于公司所有员工平均工资的员工姓名。

第3小题作答区域

4. 删除会员表中姓名为"张三"的所有记录。

第4小题作答区域

5. 查询员工人数超过 50 人的店铺名称及员工人数，并按员工人数降序排列。

第5小题作答区域

第四题 教学数据库 · 排名与复杂聚合

(25分)

对于教学数据库的四个基本表，试写出下列操作的 SQL 语句（每小题 5 分）：

1. 查询没有选修任何课程的学生学号和姓名（使用 NOT EXISTS 或 NOT IN）。

第1小题作答区域

2. 查询至少选修了学号为 s001 的同学所选的所有课程的学生学号（关系除法思想）。

第2小题作答区域

3. 查询每门课程的最高分、最低分和平均分，按课程编号升序排列。

第3小题作答区域

4. 查询选修课程数量最多的学生的学号和姓名。

第4小题作答区域

5. 查询每个学生的总成绩排名，要求并列时名次相同且后续名次连续（使用 DENSE_RANK）。

第5小题作答区域

第五题 综合新场景 · 图书馆数据库

(25分)

本题使用以下图书馆数据库（新场景，考察迁移应用能力）：

图书表 Book BookID 图书编号（主键） Title 书名 Author 作者 Price 定价 CatID 分类编号（外键）	分类表 Category CatID 分类编号（主键） CatName 分类名称
读者表 Reader RID 读者编号（主键） Rname 读者姓名 RAge 年龄	借阅表 Borrow RID 读者编号（外键） BookID 图书编号（外键） BorrowDate 借阅日期 ReturnDate 归还日期（NULL=未还）

试写出下列操作的 SQL 语句（每小题 5 分）：

1. 查询目前仍未归还图书（ReturnDate 为 NULL）的读者姓名和对应书名。

第1小题作答区域

2. 查询借阅图书数量最多的读者姓名及其借阅数量。

第2小题作答区域

3. 查询每个分类中定价最高的图书书名和定价，按定价降序排列。

第3小题作答区域

4. 将所有定价低于 30 元的图书定价提高 20%。

第4小题作答区域

5. 查询借阅过"计算机"类别所有图书的读者姓名（即该读者借阅了计算机分类下的每一本书）。

第5小题作答区域

第二部分 参考答案与得分要点解析

以下答案为推荐写法，部分题目有多种等价写法，均可得分。重点关注每题下方的得分要点。

第一题答案 教学数据库 · 多表查询

第 1 小题

▶ 参考 SQL

```
SELECT DISTINCT S.SId, S.Sname, S.Ssex  
FROM Student S  
JOIN SC ON S.SId = SC.SId;
```

JOIN Student 和 SC，有 score 记录即代表在 SC 表中存在。

DISTINCT 去除同一学生因多门课出现多次的情况。

也可用子查询：WHERE SId IN (SELECT SId FROM SC)

第 2 小题

▶ 参考 SQL

```
SELECT S.SId, S.Sname,  
ROUND(AVG(SC.score), 2) AS avg_score  
FROM Student S  
JOIN SC ON S.SId = SC.SId  
GROUP BY S.SId, S.Sname  
HAVING AVG(SC.score) >= 60  
ORDER BY avg_score DESC;
```

HAVING 不能换成 WHERE，因为 AVG 是聚合函数，必须在 HAVING 中使用。

GROUP BY 后面需列出所有非聚合字段（SId 和 Sname）。

ROUND(..., 2) 保留两位小数，更规范。

第 3 小题

▶ 参考 SQL

```
SELECT DISTINCT S.Sname
FROM Student S
JOIN SC ON S.SId = SC.SId
JOIN Course ON SC.CId = Course.CId
JOIN Teacher ON Course.TId = Teacher.TId
WHERE Teacher.Tname = '张三';
```

需连接四张表：Student → SC → Course → Teacher。

DISTINCT 避免同一学生选了张三多门课时重复出现。

第 4 小题

► 参考 SQL

```
SELECT S.SId, S.Sname,
ROUND(AVG(SC.score), 2) AS avg_score
FROM Student S
JOIN SC ON S.SId = SC.SId
WHERE S.SId IN (
SELECT SId FROM SC
WHERE score < 60
GROUP BY SId
HAVING COUNT(*) >= 2
)
GROUP BY S.SId, S.Sname
ORDER BY avg_score ASC;
```

子查询先找出不及格门数 ≥ 2 的 SId (WHERE score<60 + GROUP BY + HAVING COUNT(*) ≥ 2) 。

外层 AVG 计算所有课程均分，不只是不及格课程。

两层聚合分开写，逻辑清晰。

第 5 小题

► 参考 SQL

```
SELECT S.SId, S.Sname,  
SUM(SC.score) AS total,  
DENSE_RANK() OVER (  
ORDER BY SUM(SC.score) DESC  
) AS ranking  
FROM Student S  
JOIN SC ON S.SId = SC.SId  
GROUP BY S.SId, S.Sname  
ORDER BY total DESC;
```

DENSE_RANK() 并列后名次连续 (1,1,2,3) ; RANK() 会跳号 (1,1,3,4) , 本题不用。

窗口函数不能在 HAVING 中使用, 需在 SELECT 里嵌套 GROUP BY 结果。

第二题答案 医院数据库 · 聚合与日期

第 1 小题

▶ 参考 SQL

```
SELECT DISTINCT P.Pname
FROM Patient P
JOIN Treatment T ON P.Pno = T.Pno
WHERE T.Cno = 'c3';
```

DISTINCT 避免同一患者被医生多次诊疗时重复出现。

子查询等价写法：WHERE Pno IN (SELECT Pno FROM Treatment WHERE Cno='c3')

第 2 小题

▶ 参考 SQL

```
UPDATE Doctor
SET Dno = 'd5'
WHERE Cno = 'c4';
```

UPDATE 固定格式：UPDATE 表名 SET 字段=值 WHERE 条件。

务必写 WHERE 条件，否则修改整张表所有行。

第 3 小题

▶ 参考 SQL

```
DELETE FROM Treatment
WHERE Pno IN (
SELECT Pno FROM Patient
WHERE Pname = '张三'
);
```

不能直接 DELETE FROM Treatment WHERE Patient.Pname='张三'，跨表条件必须用子查询。

先用子查询找到张三的 Pno，再删除 Treatment 中对应记录。

第 4 小题

► 参考 SQL

```
SELECT D.Dno,  
       Dept.Dname,  
       ROUND(AVG(P.Age), 2) AS avg_age  
FROM Doctor D  
JOIN Department Dept ON D.Dno = Dept.Dno  
JOIN Treatment T ON D.Cno = T.Cno  
JOIN Patient P ON T.Pno = P.Pno  
GROUP BY D.Dno, Dept.Dname  
HAVING AVG(P.Age) > 20  
ORDER BY avg_age ASC;
```

连接四张表：Doctor→Department（取科室名）；Doctor→Treatment→Patient（取年龄）。

HAVING AVG(P.Age) > 20 过滤满足条件的科室组。

GROUP BY 需包含所有非聚合字段：Dno 和 Dname。

第 5 小题

► 参考 SQL

```
SELECT Cno, Pno, Tdate,  
       ABS(YEAR(Tdate) - 2025) AS year_diff  
FROM Treatment  
WHERE ABS(YEAR(Tdate) - 2025) = (  
SELECT MAX(ABS(YEAR(Tdate) - 2025))  
FROM Treatment  
);
```

YEAR(Tdate) 提取年份；ABS(...) 取绝对值保证结果为正数。

WHERE = MAX 子查询：先用子查询求最大年份差，再过滤等于该最大值的所有行。

若需要同时显示医生姓名和患者姓名，还需 JOIN Doctor 和 Patient。

第三题答案 商店数据库 · 更新与删除

第 1 小题

▶ 参考 SQL

```
SELECT MemberID FROM Membership WHERE StoreID = 's1'

INTERSECT

SELECT MemberID FROM Membership WHERE StoreID = 's2';

-- MySQL 兼容写法 (EXISTS)

SELECT DISTINCT m1.MemberID

FROM Membership m1

WHERE m1.StoreID = 's1'

AND EXISTS (SELECT 1 FROM Membership m2

WHERE m2.MemberID = m1.MemberID

AND m2.StoreID = 's2');
```

INTERSECT 是最简洁的标准 SQL 写法，直接求两个结果集的交集。

MySQL 不支持 INTERSECT，需用 EXISTS 或 IN 子查询替代。

两种写法在考试中均可接受。

第 2 小题

▶ 参考 SQL

```
UPDATE Employee

SET Salary = Salary * 1.1

WHERE Gender = '女';
```

乘以 1.1 等于增加 10%，不要写成 +0.1。

条件只有 Gender 字段，直接写 WHERE 即可，无需子查询。

第 3 小题

▶ 参考 SQL

```
SELECT Name
FROM Employee
WHERE Salary > (SELECT AVG(Salary) FROM Employee);
```

标量子查询：内层 SELECT AVG(Salary) 返回单个值，外层用 > 比较。
此处不需要 GROUP BY，是对整张表求均值后做比较。

第 4 小题

▶ 参考 SQL

```
DELETE FROM Member
WHERE Name = '张三';
```

Member 表中直接按姓名删除，无需子查询（条件就在本表）。
若后续还有 Membership 关联记录，需先删 Membership 再删 Member。

第 5 小题

▶ 参考 SQL

```
SELECT StoreName, EmployeeCount
FROM Store
WHERE EmployeeCount > 50
ORDER BY EmployeeCount DESC;
```

直接查 Store 表，条件 EmployeeCount > 50，按人数降序。
若题目要求实时统计而非存储值，改为子查询 COUNT(*)。

第四题答案 教学数据库 · 排名与复杂聚合

第 1 小题

▶ 参考 SQL

```
SELECT SId, Sname FROM Student
WHERE SId NOT IN (SELECT SId FROM SC);

-- NOT EXISTS 写法
SELECT SId, Sname FROM Student S
WHERE NOT EXISTS (
SELECT 1 FROM SC WHERE SC.SId = S.SId
);
```

NOT IN 和 NOT EXISTS 语义等价，选一种写即可。

注意：若 SC.SId 中有 NULL，NOT IN 可能返回空集，NOT EXISTS 更安全。

第 2 小题

▶ 参考 SQL

```
SELECT SId, Sname FROM Student
WHERE SId IN (
SELECT SId FROM SC
WHERE CId IN (
SELECT CId FROM SC WHERE SId = 's001'
)
GROUP BY SId
HAVING COUNT(DISTINCT CId) = (
SELECT COUNT(*) FROM SC WHERE SId = 's001'
)
);
```

思路：选了 s001 所有课 = 选的 s001 课程中的课程数 等于 s001 所选总课程数。

HAVING COUNT(DISTINCT CId) 确保只统计 s001 有的那些课。

这是关系除法的 SQL 实现，是高难度题型。

第 3 小题

► 参考 SQL

```
SELECT CId,  
MAX(score) AS max_score,  
MIN(score) AS min_score,  
ROUND(AVG(score), 2) AS avg_score  
FROM SC  
GROUP BY CId  
ORDER BY CId ASC;
```

直接在 SC 表上 GROUP BY CId，用 MAX/MIN/AVG 聚合。

若需要显示课程名称，还需 JOIN Course ON SC.CId = Course.CId。

第 4 小题

► 参考 SQL

```
SELECT S.SId, S.Sname  
FROM Student S  
JOIN SC ON S.SId = SC.SId  
GROUP BY S.SId, S.Sname  
HAVING COUNT(SC.CId) = (  
SELECT MAX(course_cnt)  
FROM (  
SELECT COUNT(CId) AS course_cnt  
FROM SC  
GROUP BY SId  
) t  
);
```

先用子查询找出所有学生中选课数量的最大值，再过滤等于该最大值的学生。

嵌套子查询：内层派生表 t 计算每人选课数；最外层 MAX 取最大值。

第 5 小题

► 参考 SQL

```
SELECT S.SId, S.Sname,  
SUM(SC.score) AS total,  
DENSE_RANK() OVER (  
ORDER BY SUM(SC.score) DESC  
) AS ranking  
FROM Student S  
JOIN SC ON S.SId = SC.SId  
GROUP BY S.SId, S.Sname  
ORDER BY total DESC;
```

DENSE_RANK(): 并列时名次相同，后续名次连续 (1,1,2,3)。

RANK(): 并列时名次相同，后续名次跳号 (1,1,3,4) —— 本题不用。

窗口函数在 GROUP BY 之后执行，可以对聚合结果排名。

第五题答案 图书馆数据库 · 综合应用

第 1 小题

▶ 参考 SQL

```
SELECT R.Rname, B.Title
FROM Borrow Bor
JOIN Reader R ON Bor.RID = R.RID
JOIN Book B ON Bor.BookID = B.BookID
WHERE Bor.ReturnDate IS NULL;
```

NULL 判断必须用 IS NULL，不能用 = NULL。

连接三张表：Borrow（借阅记录）→Reader（读者名）→Book（书名）。

第 2 小题

▶ 参考 SQL

```
SELECT R.Rname, COUNT(Bor.BookID) AS borrow_cnt
FROM Reader R
JOIN Borrow Bor ON R.RID = Bor.RID
GROUP BY R.RID, R.Rname
HAVING COUNT(Bor.BookID) = (
    SELECT MAX(cnt)
    FROM (
        SELECT COUNT(BookID) AS cnt
        FROM Borrow
        GROUP BY RID
    ) t
);
```

同第四题第4小题思路：子查询求最大借阅数，外层 HAVING 过滤等于最大值的读者。

若有多人并列最多，此写法会全部返回，更严谨。

第 3 小题

▶ 参考 SQL

```
SELECT C.CatName, B.Title, B.Price
FROM Book B
JOIN Category C ON B.CatID = C.CatID
WHERE B.Price = (
SELECT MAX(B2.Price)
FROM Book B2
WHERE B2.CatID = B.CatID
)
ORDER BY B.Price DESC;
```

相关子查询：内层 WHERE B2.CatID = B.CatID 关联外层，对每本书求其分类内最高价。
若一个分类有多本书并列最高价，全部返回。

第 4 小题

▶ 参考 SQL

```
UPDATE Book
SET Price = Price * 1.2
WHERE Price < 30;
```

乘以 1.2 等于提高 20%。直接在 Book 表上条件更新，无需子查询。

第 5 小题

▶ 参考 SQL

```
SELECT R.Rname
FROM Reader R
WHERE NOT EXISTS (
SELECT B.BookID
FROM Book B
JOIN Category C ON B.CatID = C.CatID
WHERE C.CatName = '计算机'
AND B.BookID NOT IN (
SELECT BookID FROM Borrow
WHERE RID = R.RID
)
);
```

关系除法经典写法：NOT EXISTS（该分类下存在未借阅的书）。

语义：不存在"计算机类图书"中有某本该读者没借过的书 = 借了所有计算机类图书。

这是最难的一题，考查关系除法，平时需要专项练习。

考点覆盖总结

题目	核心语法	难度	真题关联
第一题	JOIN / GROUP BY / HAVING / DENSE_RANK	★★★	教学库原题变体
第二题	四表JOIN / UPDATE / DELETE / YEAR() / MAX子查询	★★★	医院库原题变体
第三题	INTERSECT / EXISTS / UPDATE / 标量子查询	★★☆	商店库原题变体
第四题	NOT IN / NOT EXISTS / 嵌套子查询 / DENSE_RANK	★★★	教学库进阶题
第五题	IS NULL / 相关子查询 / 关系除法 NOT EXISTS	★★★	新场景迁移题

建议：先独立完成试题页，限时 50 分钟（模拟考试节奏），再对照答案逐题分析失分原因。
本预测题仅供参考，实际考题以官方为准 · 昆明理工大学 891计算机专业核心综合考研